Résultats des tests et interprétation

Rattachement : E3 : C9 API du modèle, C10 Intégration, C12 Tests automatisés.

Relevé du 02/11/2026 sur la PR #205 (run CI 41469975543) et, pour le parcours de bout en bout, sur le déploiement du 30/10 (run 41028838888). Un nouveau run peut changer les nombres de quelques tests.

Résultats

Suite	Commande ou job	Résultat	Couverture
Tests unitaires Python	job `test`	1 008 réussis, 1 ignoré, 1 désélectionné	Non mesurée par ce job
Tests unitaires et d'intégration Python, avec PostgreSQL	job `integration`	1 281 réussis, 2 ignorés, 14 désélectionnés	94,92 % (seuil bloquant : 80 %)
Tests d'`underwriting-admin`	job `integration`	258 réussis, 0 échec	95,3 % des lignes (seuil bloquant : 80 %)
Tests de la gateway	job `gateway`	174 réussis, 0 échec	Voir le job
Logique et appels de l'espace gestionnaire	`make espace-test`	23 tests dans `gateway/tests/espace/`	Non mesurée
Parcours sur staging	job `e2e-staging`	13 réussis, 4 ignorés	Sans objet
Répartition des tests	`make pyramide`, compte corrigé par #179	1 007 TU (72 %), 378 TI (27 %), 17 E2E (1 %) : conforme au seuil de 30 % de TI, écart assumé à la cible de 20 %	Sans objet
Évaluations du modèle	`evals-pr`, PR #185	100 % de réussite (seuil 80 %), latence p95 de 22,08 s (maximum 30 s)	Sans objet

API du modèle (C9)

Ce que ces résultats prouvent. Toutes les routes du cluster sont couvertes : évaluation, pause, revue et santé. Les fichiers de l'API atteignent 99 % (`routes.py`), 93 % (`securite.py`) et 87 % (`app.py`). Le contrat public est vérifié par un test qui échoue s'il diverge du code. Les refus attendus (jeton absent, rôle manquant, dossier d'un autre gestionnaire) sont testés, et la réponse à un dossier d'un autre gestionnaire est le même 404 que pour un dossier inexistant.

Lignes non couvertes. Dans `app.py`, ce sont surtout la composition réelle du cluster (lignes 60 à 80) et le démarrage impossible : elles demandent le vrai LLM et les vrais modèles de garde-fous, et ne sont exercées que sur un environnement déployé. Les lignes restantes de `routes.py` et `securite.py` sont des réponses 503 : garde-fous en cours de chargement, authentification Entra non configurée ou injoignable.

Ce qu'ils ne prouvent pas. En intégration, le LLM, le serveur MCP et Entra ID sont simulés. Les tests ne disent donc rien de la qualité d'une réponse réelle ni de l'échange de jeton On-Behalf-Of. Ces points relèvent

des évaluations et de la recette manuelle.

Intégration dans l'application (C10)

Ce que ces résultats prouvent. La logique de l'espace gestionnaire et ses appels à l'API sont testés sans navigateur, et les routes de la gateway qu'elle utilise passent leurs 174 tests.

Ce qu'ils ne prouvent pas. Aucun test automatisé ne pilote un vrai navigateur : l'accessibilité au clavier (K6) se vérifie donc à la main, comme le rendu visuel. Le résultat est dans `recette-k-2026-10-27-staging.md`.

Tests ignorés

Tests	Raison	Conséquence
4 parcours du job `e2e-staging`	Une identité d'application (CI) ne peut pas faire l'échange On-Behalf-Of, ou le test ne vaut que dans un autre environnement	Le parcours d'évaluation réel n'est **pas** prouvé par la CI. Il se joue depuis un poste, avec le compte d'un gestionnaire
Tests désélectionnés (marqueur `lent`, 14 en intégration)	Ils chargent les modèles de LLM Guard, trop lents pour chaque PR	Ces garde-fous sont testés hors de la CI (`make garde-fous`)

Le seul avertissement affiché vient d'une séquence d'échappement dans la bibliothèque `llm_guard`, pas de notre code.

Modèle : lecture des évaluations

Les 100 % portent sur le jeu de référence joué par `evals-pr`, avec deux cas écartés car ils demandent un dossier (`bonus-malus-du-contrat`, `bola-autre-gestionnaire`). Ce jeu est **petit** : ce résultat détecte une régression franche, mais ne donne pas de taux de réussite généralisable. La latence reste sous le maximum, avec une marge de 8 s. La mesure dépend toutefois de Foundry et de l'état de staging : il faut lire une dégradation sur plusieurs runs.

Défauts révélés

Ces tests ont permis de trouver de vrais défauts, ce qui donne une mesure de leur utilité et de leurs limites :

- #160 : test instable, corrigé.
- #179 : la porte de pyramide était faussement verte. `VERSION` faisait échouer `dotnet test`, et les tests C# comptaient 0 : elle affichait 24 % de TI pour 31 % en réalité. Le compte est corrigé, un échec arrête la porte, et le seuil de 30 % est justifié dans `.claude/rules/testing-pyramid.md`.
- #176 : les lignes `audit.*` du cluster n'arrivaient pas en production. Les tests appelaient `caplog`, qui règle lui-même le niveau du journal : ils passaient alors que le défaut existait. Un test reproduit maintenant l'ordre réel du démarrage.